# Database Systems
# Practice Examination

## Preamble

Please read the following instructions carefully:

- This exam has seven questions and 13 pages. Make sure your copy contains all exercises and all questions. The following table summarizes the questions in this exam and their weights.

| **Question** | **Points** |
|---|---|
| 1. Relational Algebra | 10 |
| 2. SQL | 20 |
| 3. ER Modeling | 16 |
| 4. Design Theory | 14 |
| 5. Physical Design | 12 |
| 6. Query Processing and Optimization | 14 |
| 7. Transactions, Concurrency Control, and Recovery | 14 |
| **Total** | **100** |

- Read the text of each exercise carefully before solving it.
- If you get stuck on an exercise, proceed with another one and come back later.
- If your solution is based on an assumption that you have made, please add a comment for clarification.
- Aids are allowed during the exam, i.e., you can consult aids such as books, notes, etc.
- As a reference, we recommend the course material (book, slides, exercises, etc.) as it complies with the notation used in the exam.
- It is NOT allowed to seek help from or consult others (human beings, artificial intelligence, etc.) during the exam.
- Good luck!

# 1 Relational Algebra (10 points)

Consider the following relations capturing information about a movie streaming service:

```
studio(sid, studioName, country, foundedYear)
movie(mid, title, genre, releaseYear, sid -> studio)
subscriber(uid, username, email, subscription)
viewing(uid -> subscriber, mid -> movie, watchDate, rating)
```

where the primary keys are underlined and the foreign keys are denoted by arrows.

Fill in the missing information that should go into the boxes to make the query compute the required information. Note that your solution should not output additional information.

To enter the operation symbols into the answer sheet, please use the following replacements:

| **Operation symbol** | **What to write in the answer sheet** |
|---|---|
| $\rho$ | rho |
| $\gamma$ | gamma |
| $\Pi$ | pi |
| $\sigma$ | sigma |
| $\bowtie$ | join |
| $\times$ | x |
| $-$ | minus |

## 1.1 (4 points)

Find all subscribers who have watched movies from studios in 'USA' and gave a rating of 5. For each such subscriber, list their uid and username.

$$\boxed{\text{Box 1}}_{\text{uid, username}} (\boxed{\text{Box 2}}_{\text{rating} = 5} (\text{subscriber} \bowtie \text{viewing} \bowtie \text{movie} \bowtie (\boxed{\text{Box 3}}_{\text{country} = \text{'USA'}} (\boxed{\text{Box 4}}))))$$

## 1.2 (6 points)

Find all subscribers who have NOT watched any Horror movie. For each such subscriber, list their uid.

$$\Pi_{\text{uid}}(\boxed{\text{Box 5}})\ \boxed{\text{Box 6}}\ \Pi_{\text{uid}}(\text{viewing} \bowtie_{\text{viewing.mid} = \text{movie.mid}} (\boxed{\text{Box 7}}_{\text{genre} = \text{'Horror'}} (\boxed{\text{Box 8}})))$$

# 2 SQL (20 points)

## 2.1 Evaluate whether the following statements about SQL are true or false.

### 2.1.1 (2 points)

The expression `NULL = NULL` evaluates to TRUE in SQL.

(a) True (b) False

### 2.1.2 (2 points)

The condition `x IN (empty set)` evaluates to FALSE, while `x > ALL (empty set)` evaluates to TRUE.

(a) True (b) False

## 2.2 Consider again the relational schema from Question 1:

```
studio(sid, studioName, country, foundedYear)
movie(mid, title, genre, releaseYear, sid -> studio)
subscriber(uid, username, email, subscription)
viewing(uid -> subscriber, mid -> movie, watchDate, rating)
```

where the primary keys are underlined and the foreign keys are denoted by arrows.

Answer the following questions:

### 2.2.1 (4 points)

Which of the following queries correctly finds the average rating per movie genre? For each genre, list the genre and the average. Choose **only one** option:

(a) SELECT genre, AVG(rating)
FROM viewing, movie
GROUP BY genre;

(b) SELECT m.genre, AVG(v.rating)
FROM viewing v JOIN movie m ON v.mid = m.mid
GROUP BY m.genre;

(c) SELECT genre, AVG(rating)
FROM viewing
GROUP BY genre;

(d) SELECT AVG(rating)
FROM viewing, movie
WHERE genre GROUP BY genre;

(e) Both (a) and (b) are correct.

(f) None of the above

### 2.2.2 (4 points)

Which of the following queries correctly finds subscribers who have watched more than 10 movies? For each such subscriber, list their uid and the count. Choose **only one** option:

(a) SELECT uid, COUNT(*)
FROM viewing
WHERE COUNT(*) > 10
GROUP BY uid;

(b) SELECT uid, COUNT(*)
FROM viewing
GROUP BY uid
HAVING COUNT(*) > 10;

(c) SELECT uid
FROM viewing
GROUP BY uid
HAVING mid > 10;

(d) SELECT uid
FROM viewing
HAVING COUNT(*) > 10;

(e) Both (a) and (b) are correct.

(f) None of the above

### 2.2.3 (4 points)

Fill in the missing information that should go into the boxes of the following query: find all studios that have at least one Action movie. For each such studio, list sid and studioName.

```
SELECT DISTINCT s.sid, s.studioName
FROM studio s
WHERE [Box 1] (
    SELECT * FROM movie m
    WHERE m.sid = s.sid [Box 2] m.genre = 'Action'
);
```

### 2.2.4 (4 points)

Consider the following query that finds subscribers who have never watched any movie:

```
SELECT s.uid, s.username
FROM subscriber s LEFT JOIN viewing v ON s.uid = v.uid
WHERE v.uid IS NULL;
```

Which of the following statements is **correct**? Choose **one or more** options:

(a) This query uses an outer join to preserve all subscribers.

(b) The WHERE clause filters for subscribers with no matching viewing records.

(c) This query is equivalent to: `SELECT uid, username FROM subscriber WHERE uid NOT IN (SELECT uid FROM viewing)`.

(d) If we replace LEFT JOIN with INNER JOIN, the query would return an empty result set (no rows).

(e) The condition `v.uid IS NULL` checks for subscribers without any viewings.

(f) All of the above.

# 3 ER Modeling (16 points)

## 3.1 Evaluate whether the following statements about ER model concepts are true or false.

### 3.1.1 (2 points)

A weak entity type is represented by a double-bordered rectangle and its identifying relationship by a double-bordered diamond.

(a) True (b) False

### 3.1.2 (2 points)

When mapping a 1:N relationship to the relational model, the foreign key is placed on the "one" side of the relationship.

(a) True (b) False

### 3.1.3 (2 points)

A composite attribute should be flattened to its simple component attributes when mapping to the relational model (the composite attribute itself is not stored).

(a) True (b) False

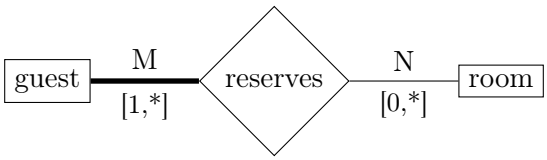
## 3.2 (4 points)

Consider the relationship *reserves* where

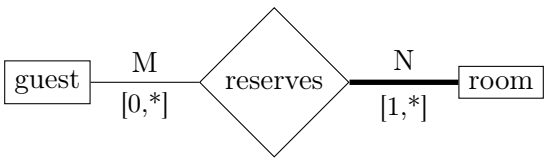
- each *room* can be reserved by many guests or no guest (partial participation)
- each *guest* must reserve at least one room (total participation)

Which of the following ER diagram representations is **correct**? Choose **only one** option:

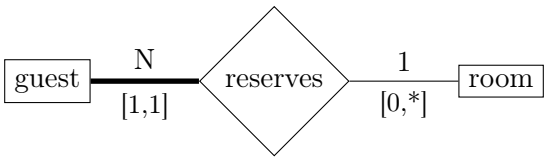
*Note: Chen notation (1, N, M) is placed near the relationship diamond. [min, max] notation is placed near the entity. Double lines indicate total participation.*



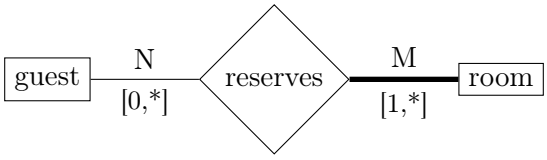

(a)




(b)




(c)

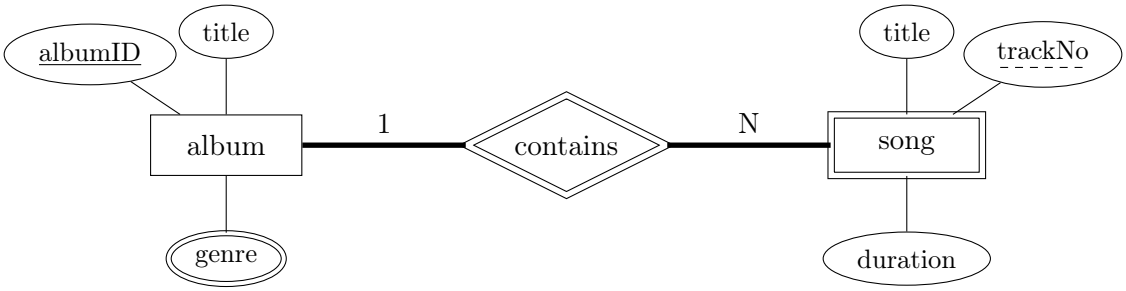



(d)

(e) None of the above

### 3.3 (6 points)

Consider the following ER diagram showing a music database with *album* (strong entity) and *song* (weak entity):




*Note: Double rectangle = weak entity. Double diamond = identifying relationship. Dashed underline on trackNo = partial key. Double ellipse on genre = multi-valued attribute. The song entity cannot exist without its owning album.*

Which combination of relations should be created for this ER diagram? Note that we aim for a minimal schema. Choose **one or more** options:

(a) album(albumID, title, genre)

(b) album(albumID, title)

(c) genre(albumID → album, genreName)

(d) song(trackNo, title, duration, albumID → album)

(e) song(albumID → album, trackNo, title, duration)

(f) contains(albumID → album, trackNo → song)

(g) song(songID, trackNo, title, duration, albumID → album)

(h) None of the above

# 4 Design Theory (14 points)

## 4.1 Evaluate whether the following statements about design theory are true or false.

### 4.1.1 (2 points)

If $X \to Y$ and $Y \to Z$ are functional dependencies in $F$, then $X \to Z$ can be derived using Armstrong's axiom of transitivity.

(a) True (b) False

### 4.1.2 (2 points)

A relation is in BCNF if and only if every non-trivial functional dependency has a superkey on the left-hand side.

(a) True (b) False

## 4.2 Consider a relation schema $R = (A, B, C, D, E)$ with the following functional dependencies (FDs):

$$
\begin{aligned}
A &\to B \\
BC &\to D \\
D &\to E \\
E &\to A
\end{aligned}
$$

Answer the following questions:

### 4.2.1 (4 points)

Which of the following are **candidate keys** of $R$? Choose **one or more** options:

(a) A

(b) C

(c) AC

(d) BC

(e) DC

(f) EC

(g) ABC

(h) None of the above

### 4.2.2 (3 points)

Which of the following statements is correct about the normalization status of relation $R$? Choose **only one** option:

(a) The relation $R$ is in BCNF.

(b) The relation $R$ is in 3NF but not in BCNF.

(c) The relation $R$ is in 2NF but not in 3NF.

(d) The relation $R$ is not in 2NF.

(e) None of the above

#### 4.2.3 (3 points)

Which of the following decompositions of the relational schema $R = (A, B, C, D, E)$ is **lossless**? Choose **one or more** options:

(a) $R_1 = (A, B, C)$ and $R_2 = (C, D, E)$

(b) $R_1 = (A, B, D)$ and $R_2 = (D, E, C)$

(c) $R_1 = (B, C, D)$ and $R_2 = (A, D, E)$

(d) $R_1 = (A, B)$ and $R_2 = (B, C, D, E)$

(e) None of the above

# 5 Physical Design (12 points)

## 5.1 Evaluate whether the following statements about B+ Trees are true or false.

### 5.1.1 (2 points)

In a B+ Tree, all data records (or pointers to data records) are stored only in the leaf nodes, while internal nodes contain only keys and child pointers.

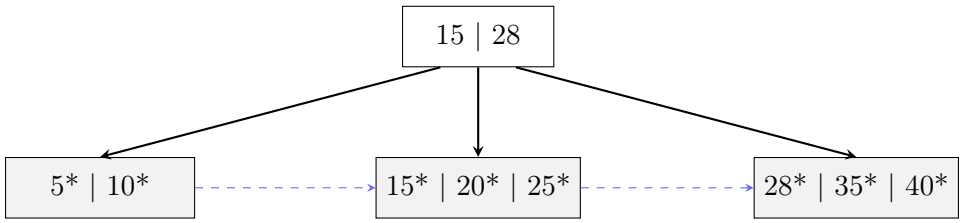
(a) True (b) False

### 5.1.2 (2 points)

For a B+ Tree with degree $d = 5$, a leaf node must contain at least 2 keys.

(a) True (b) False

## 5.2 Consider the following B+ Tree with degree $d = 4$.




Please make the following assumptions:

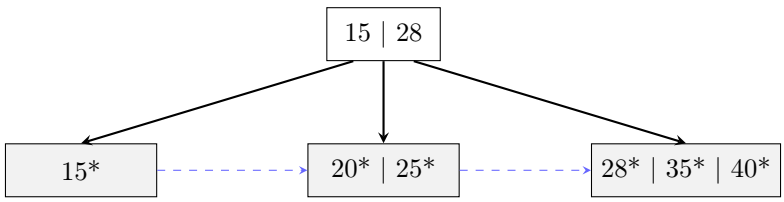
- The left pointer of a key $k$ in a non-leaf node leads towards keys less than $k$, while the right pointer leads towards keys greater than or equal to $k$.
- A leaf node underflows when the number of keys goes below $\lceil \frac{d-1}{2} \rceil$.
- An internal node underflows when the number of pointers goes below $\lceil \frac{d}{2} \rceil$.

Answer the following questions:

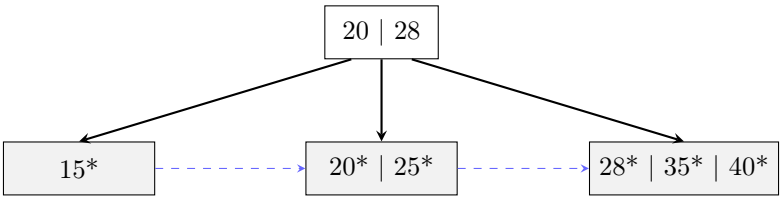
### 5.2.1 (4 points)

Delete 5*, then delete 10*. What is the resulting tree? Choose **only one** option:

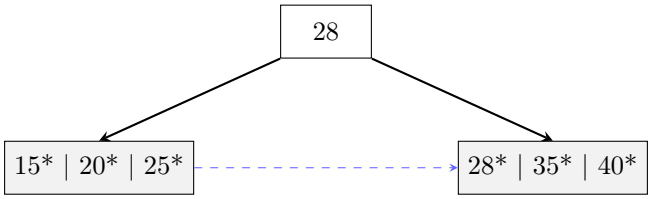
(a)



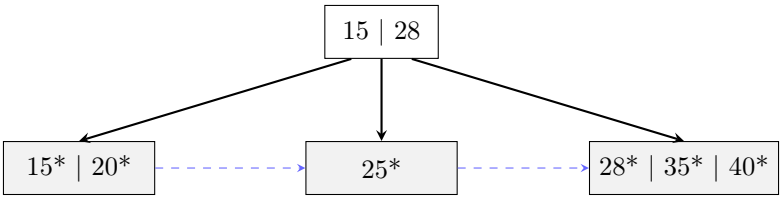
(b)




(c)




(d)

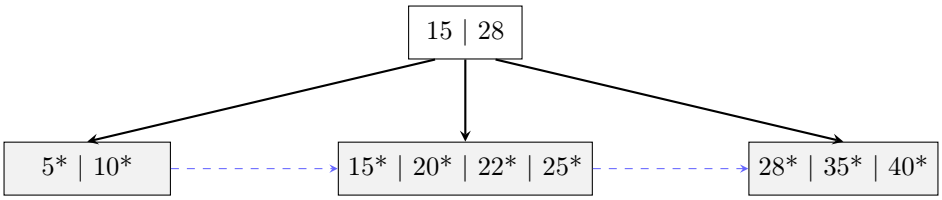



(e) None of the above

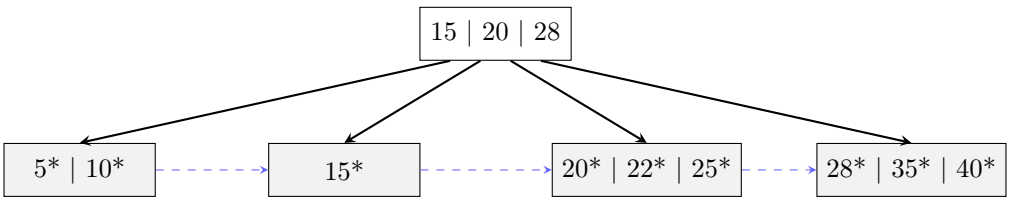
### 5.2.2 (4 points)

Consider the original tree (ignore operations in 5.2.1). Insert 22* into the tree. What is the resulting tree? Choose **only one** option:

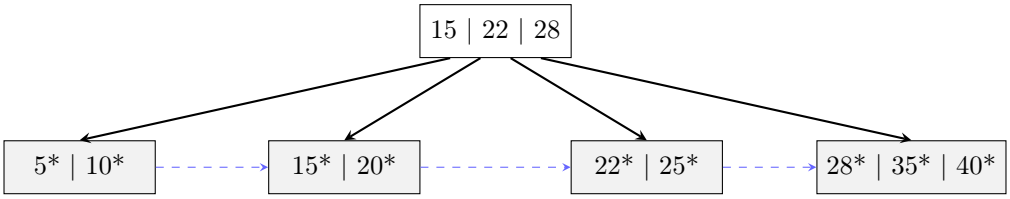
(a)



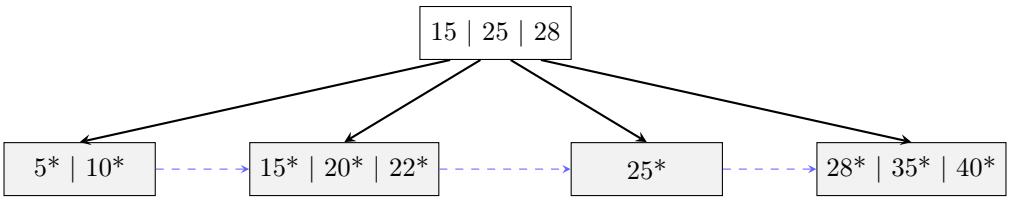

(b)



(c)




(d)




(e) None of the above

# 6 Query Processing and Optimization (14 points)

## 6.1 Evaluate whether the following statements about external sorting are true or false.

### 6.1.1 (2 points)

In 2-Way External Merge Sort, Phase 1 creates sorted runs of size equal to the number of available buffer pages, while Phase 2 merges only 2 runs at a time regardless of available buffers.

(a) True (b) False

### 6.1.2 (2 points)

In Multi-Way External Merge Sort with $B$ buffer pages, Phase 2 can merge up to $B - 1$ runs in each pass (using one buffer for output).

(a) True (b) False

## 6.2 Consider a database with relation $R$ containing 90,000 tuples, where 50 tuples fit on one page. The system has 100 buffer pages available.

Answer the following questions:

### 6.2.1 (4 points)

Suppose you need to sort relation $R$ using Multi-Way External Merge Sort. Which of the following statements is **correct**? Choose **one or more** options:

(a) The number of pages for $R$ is 1,800.

(b) Phase 1 creates 18 initial sorted runs.

(c) Phase 2 requires 1 pass to merge all runs (since $18 < 99$).

(d) The total I/O cost is $2 \times 1,800 \times 2 = 7,200$ page transfers.

(e) If we only had 10 buffer pages, Phase 1 would create 180 runs.

(f) None of the above

### 6.2.2 (2 points)

For the same relation $R$ (1,800 pages), if we use 2-Way External Merge Sort instead, how many passes are needed in Phase 2? Choose **only one** option:

(a) 11 passes (since $2^{11} = 2048 > 1800$)

(b) 18 passes

(c) 1 pass

(d) 100 passes

(e) None of the above

### 6.3 (4 points)

Consider two relations: $S$ has 4,000 pages and $T$ has 200 pages. The system has 52 buffer pages available. You need to join $S$ and $T$ using Block Nested-Loop Join.

Which of the following statements is **correct**? Choose **one or more** options:

(a) Using $T$ as the outer relation: Cost $= 200 + 4 \times 4,000 = 16,200$ page transfers.

(b) Using $S$ as the outer relation: Cost $= 4,000 + 80 \times 200 = 20,000$ page transfers.

(c) The smaller relation should always be used as the outer relation to minimize I/O cost.

(d) With $T$ as outer, we need 4 iterations because $\lceil 200/50 \rceil = 4$.

(e) The formula for Block Nested-Loop Join cost is: $|outer| + \lceil |outer|/(B-2) \rceil \times |inner|$.

(f) None of the above

# 7 Transactions, Concurrency Control, and Recovery (14 points)

## 7.1 Evaluate whether the following statements are true or false.

### 7.1.1 (2 points)

The REDO phase in log-based recovery processes the log in forward direction (from beginning to end).

(a) True (b) False

### 7.1.2 (2 points)

Deadlock can occur when two or more transactions are waiting for locks held by each other.

(a) True (b) False

## 7.2 We consider three transactions T1, T2, and T3 consisting of the following actions:

```
T1: R(M) W(N) C
T2: W(M) R(P) W(P) C
T3: R(N) W(M) R(P) C
```

R(X) represents reading object X, W(X) represents writing object X, and C represents Commit.

Consider the following two schedules:

**Schedule (1):**

| Transaction | Operations |
|---|---|
| T1 | R(M) W(N) C |
| T2 | W(M) R(P) W(P) C |
| T3 | R(N) W(M) R(P) C |

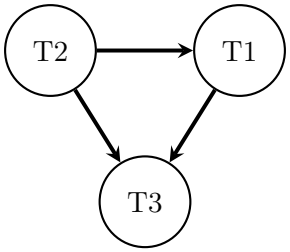
**Schedule (2):**

| Transaction | Operations |
|---|---|
| T1 | R(M) W(N) C |
| T2 | W(M) R(P) W(P) C |
| T3 | R(N) W(M) R(P) C |

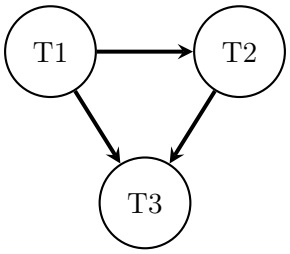
Answer the following questions:

### 7.2.1 (3 points)

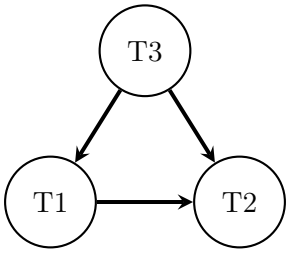
Which precedence graph of Schedule (1) is **correct**? Choose **one** option:



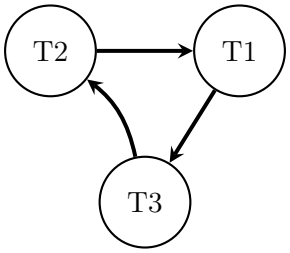

(a)




(b)




(c)




(d)

(e) None of the above

### 7.2.2 (4 points)

Which of the following statements about Schedule (1) is **correct**? Choose **one or more** options:

(a) Schedule (1) is a serial schedule.

(b) Schedule (1) is a conflict serializable schedule.

(c) Schedule (1) is a serializable schedule.

(d) The equivalent serial schedule of Schedule (1) can be T2, T1, T3.

(e) The equivalent serial schedule of Schedule (1) can be T1, T2, T3.

(f) None of the above

#### 7.2.3 (3 points)

Analyze Schedule (2). Which type(s) of schedule does Schedule (2) belong to? Choose **one or more** options:

(a) Serial schedule

(b) Serializable schedule

(c) Conflict serializable schedule

(d) None of the above

# ANSWERS SECTION

## Question 1: Relational Algebra

### 1.1 (4 points)

**Box 1:** pi (projection)

**Box 2:** sigma (selection)

**Box 3:** sigma (selection)

**Box 4:** studio

**Complete answer:** $\Pi_{\text{uid, username}}(\sigma_{\text{rating}=5}(\text{subscriber} \bowtie \text{viewing} \bowtie \text{movie} \bowtie (\sigma_{\text{country} = \text{'USA'}}(\text{studio}))))$

**Explanation:** Join all tables to connect subscribers to studios through viewings and movies, then filter for USA studios with rating = 5, and project the required attributes.

### 1.2 (6 points)

**Box 5:** subscriber

**Box 6:** - (minus / set difference)

**Box 7:** sigma (selection)

**Box 8:** movie

**Complete answer:** $\Pi_{\text{uid}}(\text{subscriber}) - \Pi_{\text{uid}}(\text{viewing} \bowtie_{\text{viewing.mid} = \text{movie.mid}} (\sigma_{\text{genre} = \text{'Horror'}}(\text{movie})))$

**Explanation:** All subscriber UIDs MINUS those who have watched Horror movies = subscribers who have NOT watched any Horror movie. This is the standard "NOT" pattern in relational algebra.

---

## Question 2: SQL

### 2.1.1 (2 points)

**Answer: (b) False**

Any comparison with NULL (including NULL = NULL) yields UNKNOWN, not TRUE. To check for NULL, you must use IS NULL or IS NOT NULL.

### 2.1.2 (2 points)

**Answer: (a) True**

x IN (empty set) is FALSE because x cannot be equal to any element in an empty set. x > ALL (empty set) is TRUE (vacuously true) because x is greater than all zero values in the empty set.

### 2.2.1 (4 points)

**Answer: (b)**

Option (b) correctly joins viewing and movie on mid and groups by genre. Option (a) is wrong because it creates a Cartesian product without proper join condition. Option (c) is wrong because viewing doesn't have a genre column. Option (d) has invalid syntax.

#### 2.2.2 (4 points)

**Answer: (b)**

Correct: GROUP BY uid, then HAVING COUNT(*) > 10 filters groups. Cannot use aggregate functions like COUNT in WHERE clause (option a). Option (c) uses non-aggregated mid in HAVING. Option (d) is missing GROUP BY.

#### 2.2.3 (4 points)

**Box 1:** EXISTS

**Box 2:** AND

**Complete query:**

```
SELECT DISTINCT s.sid, s.studioName
FROM studio s
WHERE EXISTS (
    SELECT * FROM movie m
    WHERE m.sid = s.sid AND m.genre = 'Action'
);
```

**Explanation:** EXISTS checks if the correlated subquery returns at least one row. For each studio, we check if there exists a movie record with matching sid and genre = 'Action'.

#### 2.2.4 (4 points)

**Answer: (f) All of the above**

(a) TRUE: LEFT JOIN preserves all subscribers. (b) TRUE: WHERE v.uid IS NULL filters for those without viewings. (c) TRUE: equivalent to NOT IN approach. (d) TRUE: INNER JOIN only returns matching rows, but then filtering for IS NULL means no rows match (empty result). (e) TRUE: IS NULL checks for no matching records.

---

### Question 3: ER Modeling

#### 3.1.1 (2 points)

**Answer: (a) True**

A weak entity type IS represented by a double-bordered rectangle, and its identifying relationship (the relationship that connects it to its owner entity) IS represented by a double-bordered diamond. This is standard ER notation.

#### 3.1.2 (2 points)

**Answer: (b) False**

In a 1:N relationship, the foreign key is placed on the N-side (the "many" side), NOT the "one" side. This is because each entity on the N-side references exactly one entity on the 1-side. Placing the FK on the 1-side would require storing multiple values (violating 1NF).

### 3.1.3 (2 points)

**Answer: (a) True**

Composite attributes (e.g., "address" with components street, city, zip) are flattened to their simple components when mapping to the relational model. Only the leaf-level simple attributes are stored; the composite grouping itself is not represented as a column.

### 3.2 (4 points)

**Answer: (a)**

**Interpretation of the requirements:**

- Guest [1,*]: must reserve (min=1, total participation shown by double line) at least one room (max=*)
- Room [0,*]: can be reserved by none (min=0, partial participation) or many guests (max=*)
- Chen notation: M near guest (many guests), N near room (many rooms) - this is M:N

**Why other options are wrong:**

- (b): Participation swapped - would mean rooms must be reserved (total) and guests are optional (partial)
- (c): Shows N:1 relationship where guest reserves exactly one room [1,1] - wrong cardinality
- (d): Participation completely swapped between entities

### 3.3 (6 points)

**Answer: (b), (c), (e)**

**Correct mapping (3 relations):**

- **(b)** album(<u>albumID</u>, title)
  - Strong entity with simple attributes only
  - genre is multi-valued → separate table (NOT included here)
- **(c)** genre(<u>albumID → album, genreName</u>)
  - Multi-valued attribute creates separate table
  - Composite PK: albumID + genreName (both underlined)
- **(e)** song(<u>albumID → album, trackNo</u>, title, duration)

  - Weak entity mapping: owner's PK (albumID) + partial key (trackNo) = composite PK
  - albumID is both FK and part of PK
  - All simple attributes of song included

**Why other options are wrong:**

- (a): genre should NOT be in album table (it's multi-valued)
- (d): trackNo alone cannot be PK for weak entity (needs owner's PK too)
- (f): No separate "contains" table needed - weak entity absorbs the identifying relationship
- (g): Weak entities don't get their own independent ID; they use owner's PK + partial key

---

## Question 4: Design Theory

### 4.1.1 (2 points)

**Answer: (a) True**

Armstrong's axiom of transitivity states: If $X \to Y$ and $Y \to Z$, then $X \to Z$. This is one of the three fundamental axioms (Reflexivity, Augmentation, Transitivity) that form the basis for deriving all valid functional dependencies.

### 4.1.2 (2 points)

**Answer: (a) True**

This is the exact definition of BCNF. A relation is in BCNF if and only if for every non-trivial FD $X \to Y$, the left-hand side $X$ is a superkey. Unlike 3NF, BCNF has no exception for prime attributes on the RHS.

### 4.2.1 (4 points)

**Answer: (c), (d), (e), (f)**

First, identify that C never appears on RHS of any FD, so C must be in every candidate key.

Computing closures:

- $(AC)^+ = \{A, C, B, D, E\}$ (using $A \to B$, $BC \to D$, $D \to E$) - **candidate key!**
- $(BC)^+ = \{B, C, D, E, A\}$ (using $BC \to D$, $D \to E$, $E \to A$) - **candidate key!**
- $(DC)^+ = \{D, C, E, A, B\}$ (using $D \to E$, $E \to A$, $A \to B$) - **candidate key!**
- $(EC)^+ = \{E, C, A, B, D\}$ (using $E \to A$, $A \to B$, $BC \to D$) - **candidate key!**
- A alone: $A^+ = \{A, B\}$ - not a superkey (missing C, D, E)
- C alone: $C^+ = \{C\}$ - not a superkey
- ABC is a superset of AC, so it's a superkey but NOT minimal (not a candidate key)

#### 4.2.2 (3 points)

**Answer: (b) 3NF but not BCNF**

Candidate keys: AC, BC, DC, EC. Prime attributes: A, B, C, D, E (ALL are prime!)

Check BCNF: For $A \to B$, A is not a superkey ($A^+ = \{A, B\}$). Violates BCNF.

Check 3NF: For each FD, either LHS is superkey OR RHS is prime:

- $A \to B$: A not superkey, but B is prime (part of key BC) ✓
- $BC \to D$: BC is superkey ✓
- $D \to E$: D not superkey, but E is prime (part of key EC) ✓
- $E \to A$: E not superkey, but A is prime (part of key AC) ✓

All FDs satisfy 3NF condition, so relation is in 3NF but not BCNF.

#### 4.2.3 (3 points)

**Answer: (b), (c)**

For lossless decomposition: $(R_1 \cap R_2)^+$ must contain $(R_1 - R_2)$ or $(R_2 - R_1)$.

(a) $R_1 \cap R_2 = \{C\}$. $C^+ = \{C\}$. Doesn't contain $\{A, B\}$ or $\{D, E\}$. **NOT lossless.**

(b) $R_1 \cap R_2 = \{D\}$. $D^+ = \{D, E, A, B\}$. Contains $R_1 - R_2 = \{A, B\}$! **Lossless!**

(c) $R_1 \cap R_2 = \{D\}$. $D^+ = \{D, E, A, B\}$. Contains $R_2 - R_1 = \{A, E\}$! **Lossless!**

(d) $R_1 \cap R_2 = \{B\}$. $B^+ = \{B\}$. Doesn't contain $\{A\}$ or $\{C, D, E\}$. **NOT lossless.**

---

### Question 5: Physical Design

#### 5.1.1 (2 points)

**Answer: (a) True**

In B+ trees, all data records (or pointers to data records) are stored only in the leaf nodes. Internal nodes contain only keys (used as separators/guides for navigation) and child pointers. This is the fundamental distinction between B+ trees and B-trees.

#### 5.1.2 (2 points)

**Answer: (a) True**

For degree $d = 5$: minimum keys in leaf $= \lceil (d-1)/2 \rceil = \lceil (5-1)/2 \rceil = \lceil 4/2 \rceil = 2$.

#### 5.2.1 (4 points)

**Answer: (c)**

**Step-by-step analysis:**

**Initial tree (d=4):** Root [15 | 28] with leaves [5* 10*], [15* 20* 25*], [28* 35* 40*].

**For d=4:** min leaf keys = $\lceil (4-1)/2 \rceil = 2$, max leaf keys = $d - 1 = 3$.

**Delete 5*:** Leaf becomes [10*] (1 key < 2 minimum → underflow!). Right sibling [15* 20* 25*] has 3 keys (more than minimum 2), so we can **redistribute**. Borrow 15* from sibling: left leaf becomes [10* 15*], sibling becomes [20* 25*]. Update separator key in root: first key of new right leaf = 20. Root becomes [20 | 28].

**Delete 10*:** Leaf [10* 15*] becomes [15*] (1 key < 2 minimum → underflow!). Right sibling [20* 25*] has only 2 keys (exactly minimum), cannot redistribute. Must **merge**: combine [15*] with [20* 25*] → [15* 20* 25*] (3 keys, valid). Remove separator 20 from root. Root becomes [28] with 2 leaves.

**Final result:** Root [28] with leaves [15* 20* 25*] and [28* 35* 40*]. This matches option (c).

**Why other options are wrong:**

- (a) Wrong separator (15 should not be in root after merge)
- (b) Wrong separator and structure
- (d) Shows underflow not handled (leaf with only 1 key)

### 5.2.2 (4 points)

**Answer: (c)**

**Step-by-step analysis:**

**Initial tree (d=4):** Root [15 | 28] with leaves [5* 10*], [15* 20* 25*], [28* 35* 40*].

**Insert 22*:** Goes to leaf [15* 20* 25*]. After insertion: [15* 20* 22* 25*] (4 keys, but max = $d - 1 = 3$ → overflow!)

**Split leaf:**

- Left leaf gets $\lfloor 4/2 \rfloor = 2$ keys: [15* 20*]
- Right leaf gets $\lceil 4/2 \rceil = 2$ keys: [22* 25*]
- **Copy up** first key of right leaf = 22

**Update parent:** Insert key 22 into root [15 | 28]. Root becomes [15 | 22 | 28] with 4 children (valid for d=4, max children = 4).

**Final result:** Root [15 | 22 | 28] with leaves: [5* 10*], [15* 20*], [22* 25*], [28* 35* 40*]. This matches option (c).

---

## Question 6: Query Processing and Optimization

### 6.1.1 (2 points)

**Answer: (a) True**

This statement correctly describes 2-Way External Merge Sort:

- Phase 1: Read $B$ pages into memory, sort them, write out a sorted run of size $B$ pages

- Phase 2: Merge exactly 2 runs at a time (regardless of how many buffers are available)
- This is why it's called "2-Way" – it only merges 2 runs per pass

### 6.1.2 (2 points)

**Answer: (a) True**

In Multi-Way External Merge Sort with $B$ buffer pages:

- We need 1 buffer page for output
- The remaining $B - 1$ buffer pages can each hold one page from a different run
- Therefore, we can merge up to $B - 1$ runs simultaneously in each pass

### 6.2.1 (4 points)

**Answer: (a), (b), (c), (d), (e)**

**Given:** 90,000 tuples, 50 tuples/page, 100 buffer pages

(a) Pages for $R$: $90,000/50 = 1,800$ pages. **Correct!**

(b) Phase 1 runs: $\lceil 1,800/100 \rceil = 18$ runs. **Correct!**

(c) Phase 2: With 100 buffers, we can merge $100 - 1 = 99$ runs per pass. Since $18 < 99$, we need only 1 pass. **Correct!**

(d) Total I/O $= 2 \times \text{pages} \times (1 + \text{Phase 2 passes}) = 2 \times 1,800 \times (1 + 1) = 7,200$ page transfers. **Correct!**

(e) With 10 buffers: $\lceil 1,800/10 \rceil = 180$ runs. **Correct!**

### 6.2.2 (2 points)

**Answer: (a)**

2-Way External Merge Sort Phase 2 passes formula: $\lceil \log_2 N_0 \rceil$ where $N_0$ = number of initial sorted runs = number of pages (in 2-Way, each page becomes one initial run)

$\lceil \log_2 1,800 \rceil = 11$ passes (since $2^{10} = 1024 < 1800$ and $2^{11} = 2048 > 1800$)

### 6.3 (4 points)

**Answer: (a), (b), (c), (d), (e)**

**Given:** $|S| = 4,000$ pages, $|T| = 200$ pages, $B = 52$ buffers

**Block Nested-Loop Join formula:** Cost $= |outer| + \lceil |outer|/(B-2) \rceil \times |inner|$

(a) $T$ as outer: $200 + \lceil 200/50 \rceil \times 4,000 = 200 + 4 \times 4,000 = 16,200$. **Correct!**

(b) $S$ as outer: $4,000 + \lceil 4,000/50 \rceil \times 200 = 4,000 + 80 \times 200 = 20,000$. **Correct!**

(c) Since $16,200 < 20,000$, using smaller relation as outer minimizes I/O. **Correct!**

(d) $\lceil 200/50 \rceil = 4$ iterations when $T$ is outer. **Correct!**

(e) The formula is correct. **Correct!**

## Question 7: Transactions, Concurrency Control, and Recovery

### 7.1.1 (2 points)

**Answer: (a) True**

REDO processes log forward (beginning to end) to reapply committed changes.

### 7.1.2 (2 points)

**Answer: (a) True**

Deadlock occurs when transactions form a cycle in the wait-for graph, each waiting for locks held by others.

### 7.2.1 (3 points)

**Answer: (a)**

Conflicts in Schedule (1):

- T2: W(M) before T1: R(M) → T2 → T1
- T1: W(N) before T3: R(N) → T1 → T3
- T2: W(P) before T3: R(P) → T2 → T3

Graph: T2 → T1, T1 → T3, T2 → T3

### 7.2.2 (4 points)

**Answer: (b), (c), (d)**

(b) Acyclic precedence graph means conflict serializable. **True!**

(c) All conflict serializable schedules are serializable. **True!**

(d) Topological sort gives: T2, T1, T3. **True!**

### 7.2.3 (3 points)

**Answer: (b), (c)**

Conflicts in Schedule (2):

- T1: R(M) before T2: W(M) → T1 → T2
- T2: W(M) before T3: W(M) → T2 → T3
- T1: W(N) before T3: R(N) → T1 → T3
- T2: W(P) before T3: R(P) → T2 → T3

The precedence graph is: T1 $\rightarrow$ T2, T2 $\rightarrow$ T3, T1 $\rightarrow$ T3.

This is acyclic! So it IS conflict serializable and serializable. Equivalent serial order: T1, T2, T3.